

Using Faiss-CPU for Efficient Vector Search: A Comprehensive Guide

Overview of Faiss-CPU

The **Facebook AI Similarity Search (Faiss)** library is a popular toolkit for efficient similarity search of dense vectors. The official `faiss-cpu` Python package is a CPU-only version of Faiss that can be installed via `pip` (`pip install faiss-cpu`). It provides a rich set of data structures (indexes) and algorithms for nearest neighbor search on CPU architectures. In this guide, we will focus exclusively on using Faiss on CPUs – covering the major index types, how to use them, best practices for conversion from GPU-based workflows, and specific guidance for retrieval-augmented generation (RAG) use cases. All examples and discussions assume the use of the **CPU version of Faiss** and avoid GPU or NVIDIA cuVS-specific functionality.

Similarity Metrics: Faiss indexes can operate on either **Euclidean distance (L2)** or **inner product (dot product)** as the similarity metric. Generally, **L2 distance** is used for measuring dissimilarity (smaller distance = more similar), while **inner product** is used for measuring similarity (higher score = more similar). In Faiss, index classes often have suffix `L2` or `IP` to denote the metric. *For example, `IndexFlatL2` uses Euclidean distance, and `IndexFlatIP` uses inner product.* When using inner product for cosine similarity, it's common to **normalize vectors to unit length** before indexing or querying (so that inner product corresponds to cosine similarity). Faiss also provides a built-in normalization transform for convenience (discussed later).

Installation: Ensure you have installed the CPU version of Faiss. For example:

```
pip install faiss-cpu
```

This will install the `faiss` module with CPU support only. Throughout this guide, we assume `import faiss` refers to this CPU version.

Major FAISS Index Types on CPU

Faiss provides multiple index types on CPU, each with different trade-offs between accuracy, speed, memory usage, and functionality. Below we detail the major index classes available in `faiss-cpu`, their characteristics, and usage patterns.

Flat Indexes (Brute-Force Search)

Flat indexes perform brute-force nearest neighbor search by comparing a query vector to *all* vectors in the database. They guarantee exact results (the true nearest neighbors) at the cost of linear scan time. Flat

indexes do **no compression or approximation** – they store and compare full vectors directly ¹ ². This makes them simple to use and exact, but potentially slow on large datasets.

- **Classes:** `faiss.IndexFlatL2` (for L2 Euclidean distance), `faiss.IndexFlatIP` (for inner product).
- **No Training Needed:** Flat indexes do not require a training phase since they don't build any centroids or quantizers. You can add vectors and query immediately.
- **Adding Vectors:** You can add vectors with `index.add(x)` where `x` is a NumPy array of shape `(n_vectors, dim)` (float32 dtype). Flat indexes **do not** support `add_with_ids` natively (because they assume implicit sequential IDs) ³. If you need custom IDs, use an `IndexIDMap` wrapper (discussed later).
- **Searching:** Use `D, I = index.search(q, k)` where `q` is a query matrix `(n_queries, dim)` and `k` is the number of nearest neighbors to find. This returns distance matrix `D` and index matrix `I` (each of shape `(n_queries, k)`). Distances will be L2 or inner product scores depending on the index type.
- **Exactness:** Flat indexes guarantee finding the true nearest neighbors (no approximation) ⁴. The trade-off is speed – scanning all vectors for each query is expensive for large databases. Faiss uses optimized BLAS and multi-threading to mitigate this, but it remains linear in database size.
- **Use Cases:** Best for **small datasets or strict accuracy requirements**. For example, if you have <100k vectors or can tolerate higher query latency, a flat index is simple and effective ⁵ ². It's also often used to establish a baseline for approximate methods.
- **Pros & Cons:** Pro: exact results and simplest usage. Con: slow for large `ntotal` (number of stored vectors), memory scales linearly with dataset (stores full vectors).

Code Example – Using a Flat Index (L2)

```
import numpy as np
import faiss

# Create some sample data
d = 128                                # dimension of vectors
np.random.seed(0)
data = np.random.rand(10000, d).astype('float32') # 10k random vectors as example

# 1. Initialize a FlatL2 index and add data
index_flat = faiss.IndexFlatL2(d)      # L2 metric
index_flat.add(data)                   # add vectors to the index (ntotal = 10000)

# 2. Perform a search for the 5 nearest neighbors of the first query vector
query_vec = data[0:1]                 # take first vector as a query (shape (1, d))
k = 5
D, I = index_flat.search(query_vec, k)
```

```
print("Nearest neighbors (indices):", I)
print("Distances:", D)
```

This code creates an `IndexFlatL2`, adds 10,000 vectors, and then searches for the 5 nearest neighbors of the first vector. Because it's a flat index, it will consider all 10k vectors to find the exact neighbors. (In practice, the output will show index 0 as the closest neighbor with distance 0, since we queried a vector that is itself in the index.)

Note on Inner Product: If using `IndexFlatIP`, you might want to normalize vectors first for cosine similarity. You can manually normalize (`x /= np.linalg.norm(x, axis=1, keepdims=True)`) or use Faiss's `faiss.normalize_L2(x)` utility to scale each vector to unit length. Alternatively, use Faiss's `IndexFlatIP` directly on un-normalized embeddings if the embedding model produces dot-product-compatible features (just remember the returned "distances" are actually **inner product scores**, higher is better in that case, whereas with L2 smaller is better).

ID Management: As mentioned, flat indexes use implicit integer IDs (0 to `ntotal-1` by insertion order). If you need to associate vectors with external IDs (like document IDs in a RAG system), you can wrap the flat index with `IndexIDMap` to enable `add_with_ids` and store a custom ID for each vector. We will discuss `IndexIDMap` in detail later.

Removing Vectors: Flat indexes support removal of vectors via `remove_ids()`, but note that this **shifts the IDs** of subsequent vectors since flat indexes store data in a simple array ⁶. Removing an element *i* will cause all elements with IDs > *i* to have their ID decremented by 1. If maintaining external references to IDs, consider using an ID map or avoid removal by rebuilding indexes if possible.

IVF Indexes (Inverted File for Approximate Search)

Inverted File Indexes (IVF) partition the vector space into clusters (cells) and restrict search to a few relevant clusters to improve speed ⁷ ⁸. They sacrifice some recall (approximation) in exchange for large speedups on bigger datasets. IVF is a two-stage process: a **coarse quantizer** (e.g., k-means centroids) first assigns each vector to a cluster, and at query time only the closest clusters to the query are searched.

- **Classes:** `faiss.IndexIVFFlat`, `faiss.IndexIVFPQ`, `faiss.IndexIVFScalarQuantizer` (IVFSQ), etc. The prefix `IndexIVF` denotes an inverted index. The variants differ in how they *store vectors in each cell*:
- **IVFFlat** stores full vectors in each list (no compression apart from clustering).
- **IVFPQ** stores vectors as PQ codes (product quantization) within each list, providing compression.
- **IVFSQ** stores vectors with scalar quantization (e.g., 8-bit or 4-bit quantization per component) ⁹.
- **Coarse Quantizer:** When creating an IVF index, you must provide a quantizer index and specify `nlist` (the number of clusters, i.e., the number of inverted lists) ¹⁰. Typically, the quantizer is a flat index over the same vector space (e.g., `IndexFlatL2`) which will hold the `nlist` centroids after training. In practice, you often use `faiss.IndexFlatL2(d)` or `IndexFlatIP(d)` as the coarse quantizer for an IVF index.
- **Training: All IVF indexes require training** before you can add vectors. The training step usually involves running k-means to find cluster centroids (and possibly additional training for PQ or SQ codebooks). You call `index.train(training_vectors)` with a representative set (often the

actual dataset or a large sample). Training will populate the coarse quantizer's centroids (and PQ codebooks if applicable).

- **Adding Vectors:** After training, use `index.add(x)` or optionally `index.add_with_ids(x, ids)`. IVF indexes *do* support `add_with_ids` natively and always store vector IDs alongside the vectors in each list ¹¹. (If you don't provide IDs, Faiss assigns them sequentially as usual.)
- **Search Parameters:** The key search-time parameter for IVF is `nprobe`, which is how many clusters (inverted lists) to search at query time ¹² ¹³. By default, `nprobe=1` (only search the single closest cluster to the query). Increasing `nprobe` trades more computation (more clusters searched) for higher recall. You can set `index.nprobe = value` before querying, or use the `ParameterSpace` / `SearchParameters` mechanism to adjust it per query (covered later). Typical values: `nprobe` in the range of 4 to 32 for many applications, but it can be higher for better accuracy at cost of speed.
- **Memory/Accuracy Trade-offs:** `nlist` (**number of clusters**) influences the trade-off between pre-processing vs search cost. A larger `nlist` means finer partitioning (fewer vectors per list on average), so queries examine a smaller fraction of the database – faster search but potentially lower recall if `nprobe` is small ¹⁴. A smaller `nlist` means each list is larger (more candidates to scan per list) – slower search but higher likelihood the true neighbor lies in the searched lists. A common heuristic is to set `nlist  $\approx$   $\sqrt{N}$` (where N is the number of vectors) ¹⁴, then tune `nprobe` for recall.
- **IVF Index Creation:** For example, to create an IVF index:

```
quantizer = faiss.IndexFlatL2(d)                # coarse
quantizer (L2)
nlist = 100                                     # number of
clusters
index_ivf = faiss.IndexIVFFlat(quantizer, d, nlist, faiss.METRIC_L2)
```

This sets up an `IndexIVFFlat` with 100 clusters using L2 distance ¹⁵ ¹⁶. You would then call `index_ivf.train(data)` before adding vectors.

- **IVFPQ and IVFSQ:** These are variants that compress the stored vectors:
- **IVFPQ:** Each vector is encoded by product quantization into a fixed-size code (e.g., 8 bytes or 16 bytes). You specify parameters `m` (number of subquantizers) and `nbits` (bits per subvector code) when constructing, e.g. `faiss.IndexIVFPQ(quantizer, d, nlist, m, nbits)`. Product quantization significantly reduces memory and speeds up distance calculations by using precomputed tables ¹⁷ ¹⁸. However, it introduces quantization error, so results are approximate. This is a good choice when you have **large datasets (millions of vectors) and need to dramatically reduce memory** with minimal loss in recall.
- **IVFSQ:** Uses scalar quantization per dimension (e.g., 8-bit) instead of PQ. Constructed via `faiss.IndexIVFScalarQuantizer(quantizer, d, nlist, qtype, metric)` where `qtype` might be `faiss.ScalarQuantizer.QT_8bit`, `QT_4bit`, etc. This compresses each component

of vectors to a lower precision ¹⁹. It's simpler but often less compact/accurate than PQ for high dims.

- **Pros & Cons:** IVF indexes excel at **speed on large datasets**. By searching only a fraction of vectors (proportional to `nprobe/nlist`), they can answer queries much faster than brute force. They also scale well – Faiss can handle millions to billions of vectors with IVF by using more clusters and possibly multi-index or sharding. The downsides: they are **approximate** (you might miss some true nearest neighbors if they fall in an un-searched cluster) and **need tuning** (choosing `nlist` and `nprobe` appropriately). They also require a one-time training cost.

- **Use Cases:** IVF is a good default for **medium to large scale** (say, >100k vectors) when exact search is too slow. For instance, in a RAG system with a few million embeddings, an `IndexIVFFlat` or `IndexIVFPQ` can give low query latency with high recall if tuned well. If memory is not tight and accuracy is priority, `IVFFlat` (no compression) with a decent `nprobe` might be best. If memory is a concern, `IVFPQ` lets you compress vectors (e.g., 16 bytes per vector) at the cost of some accuracy.

Code Example – Creating and Using an IVF Index

Below, we create an IVF index, train it, add vectors, and search it. We'll use `IndexIVFFlat` for simplicity (no PQ compression) and L2 distance.

```
import numpy as np
import faiss

d = 128
np.random.seed(42)
xb = np.random.random((20000, d)).astype('float32') # 20k training vectors
xq = np.random.random((5, d)).astype('float32')      # 5 query vectors

# 1. Create coarse quantizer and IVF index
nlist = 256
quantizer = faiss.IndexFlatL2(d) # coarse quantizer: flat L2 index
index_ivf = faiss.IndexIVFFlat(quantizer, d, nlist, faiss.METRIC_L2)

# 2. Train the IVF index (needed for clustering centroids)
index_ivf.train(xb)
print(f"Index trained: {index_ivf.is_trained}") # should be True after training

# 3. Add vectors to the index
index_ivf.add(xb)
print(f"Index size (ntotal): {index_ivf.ntotal}") # should be 20000

# 4. Configure search parameters and perform a search
index_ivf.nprobe = 16 # search in 16 out of 256 clusters
```

```

k = 5
D, I = index_ivf.search(xq, k)
print("IVF Search results (indices):\n", I)
print("IVF Search results (distances):\n", D)

```

In this example, we used `nlist=256` clusters for 20k vectors (which is a bit high but demonstrates usage). We set `nprobe=16` to search 16 clusters for each query. The `IndexIVFFlat` will typically retrieve approximate neighbors. By increasing `nprobe` (up to 256 in this case), we would get closer to exact results at the cost of more comparisons¹³²⁰. If we set `nprobe = nlist` (256), the IVF index essentially becomes a flat index (searching all clusters, i.e., exhaustive search).

Tuning IVF: In practice, you should experiment with `nlist` and `nprobe`. A common approach is to fix `nlist` such that each inverted list has a manageable number of vectors (e.g., a few hundred), and then adjust `nprobe` to balance speed/accuracy. For example, if you have 1,000,000 vectors, `nlist=1000` (so ~1000 vectors per list on average) is a starting point. If recall is low, increase `nprobe` or increase `nlist` and retrain. Faiss's `ParameterSpace` can even help systematically explore combinations (described in a later section).

IVFPQ Usage: To briefly illustrate an IVFPQ, suppose we want to compress each vector into 8 bytes using product quantization with `m=16` subquantizers and `nbits=4` bits each (4 bits * 16 = 64 bits = 8 bytes). We could do:

```

m = 16          # number of sub-quantizers
nbits = 4       # bits per sub-vector (4 bits gives 16 centroids per subvector)
index_ivfpq = faiss.IndexIVFPQ(quantizer, d, nlist, m, nbits)
index_ivfpq.train(xb)
index_ivfpq.add(xb)
index_ivfpq.nprobe = 16
D, I = index_ivfpq.search(xq, k)

```

The `IndexIVFPQ` will use more memory for precomputed tables during search but each stored vector is only 8 bytes (plus quantizer overhead) – a huge win for memory if `d=128` (which would be 512 bytes per vector in full precision). The search will compute approximate distances using PQ codes. One can also perform a re-ranking (refinement) of the results by comparing original vectors if higher accuracy is needed (see `IndexRefineFlat` below).

HNSW Index (Hierarchical Navigable Small Worlds Graph)

The **HNSW** index is a graph-based approximate nearest neighbor method that is very popular due to its strong accuracy-speed balance on CPUs. Faiss implements HNSW on the CPU as `IndexHNSW*` classes. HNSW builds a navigable small-world graph of the vectors: each vector is a node connected to its `M` nearest

neighbors, and the search proceeds by “hopping” through these links to find close items. HNSW often achieves high recall with low latency and without a clustering step.

- **Classes:** `faiss.IndexHNSWFlat` (HNSW on top of an uncompressed flat storage), `faiss.IndexHNSWPQ` (HNSW on top of PQ-compressed vectors), `faiss.IndexHNSWSQ` (HNSW with scalar-quantized vectors), etc. Most commonly, you’ll use `IndexHNSWFlat(d, M)` for dense data when memory allows storing full vectors ²¹.
- **Parameters:** Key HNSW parameters are:
 - **M:** the number of bi-directional links per node to keep in the graph ²². Higher M means each node is connected to more neighbors, which increases accuracy (and index size) at the cost of slower indexing and more memory usage. Typical M might be 16, 32, or higher for very high accuracy needs. Memory usage of HNSW is roughly $M * 2 * 4$ bytes per vector (for storing neighbor links) plus the vector storage ²³.
 - **efConstruction:** the size of the dynamic list for neighbors during the **index build** (graph construction). Higher efConstruction leads to a better graph (more accurate search) but slower indexing time ²⁴. This is often set equal or higher than M; e.g., M=32, efConstruction=64 or 100.
 - **efSearch:** the size of the dynamic list for neighbors during **query time**. This is analogous to “search breadth” – a higher efSearch means the algorithm explores more of the graph and yields higher recall at the cost of latency ²⁴. You can set `index.hnsw.efSearch` before querying, or tune it per query via `ParameterSpace`. Typical values are efSearch in the range of 20 to a few hundred depending on the trade-off needed.
 - **No Training Required:** HNSW does not need a separate training phase. The graph is built incrementally as vectors are added (each new vector is connected to its M nearest existing vectors as determined by a greedy search in the current graph).
 - **Adding Vectors:** Use `index.add(x)` to add vectors. You can add in batches or one by one. Note that **HNSW indices currently support only sequential ids (no `add_with_ids`)** – if you call `add_with_ids` on an `IndexHNSWFlat`, it will throw an error (because the underlying Flat index doesn’t support it) ²⁵. If custom IDs are needed with HNSW, wrap it with `IndexIDMap` similarly to a flat index.
 - **Searching:** Use `index.search(q, k)` as usual. You may want to adjust `efSearch` for a recall/speed trade-off. By default, `efSearch` might be set to a value like 20 or 40. Setting `efSearch = k` is a common baseline (ensuring at least k neighbors are considered), but often you use a multiple of k for higher recall (e.g., if k=10, try efSearch=50 or 100).
 - **Memory and Speed:** HNSW is quite memory-intensive (storing the graph connectivity). It is fast at query time, often outperforming IVF for high recall because it doesn’t require scanning too many vectors – it navigates the graph structure. The accuracy can be very high, often close to exact if parameters are high. However, building the HNSW graph can be slow for large datasets (as it effectively does many neighbor searches to place each point).
 - **No Deletions:** Faiss HNSW does **not support deletion of individual vectors** once added ²⁶ ²⁷. Removing a vector would break the graph links. If you need dynamic updates (addition and deletion), IVF or other approaches might be more suitable, or you would need to reconstruct the index when data changes.
 - **Use Cases:** HNSW is an excellent choice for **moderate-scale** datasets where memory is available. For example, for a few hundred thousand vectors, an `IndexHNSWFlat` (with M ~32) can give very fast searches (<10ms) with high accuracy. Even at the million-scale, HNSW is feasible on a single machine if you have enough RAM. HNSW is often the go-to for CPU ANN when you want a balance of *good recall* and *low latency* without too much parameter tuning – it tends to work well out-of-the-box.

Code Example – Using HNSW Index

```
import faiss

d = 128
data = np.random.random((5000, d)).astype('float32')  # 5k vectors
query = data[0:2]  # 2 query vectors

M = 32
index_hnsw = faiss.IndexHNSWFlat(d, M)  # HNSW on Flat (L2 by default)
# Set HNSW construction/search parameters
index_hnsw.hnsw.efConstruction = 40
# larger = better graph quality (slower build)
index_hnsw.hnsw.efSearch = 16  # could increase for higher recall

index_hnsw.add(data)  # add all vectors
print(f"HNSW index size: {index_hnsw.ntotal}")

k = 5
D, I = index_hnsw.search(query, k)
print("HNSW neighbors for queries:\n", I)
```

In this snippet, we create an `IndexHNSWFlat` with `M=32` neighbors. We set `efConstruction` to 40 before adding vectors to control graph build quality, and set a relatively small `efSearch` (16) for query time. We add 5000 vectors and then retrieve 5-NN for two queries. If we increased `efSearch`, we'd likely get more accurate results at the cost of a bit more computation. HNSW uses internal multi-threading for adding and searching; it will try to use multiple cores by default.

Important: HNSW indexes in Faiss **do not support** `add_with_ids` **directly**, so to maintain a mapping to external IDs, use `faiss.IndexIDMap(index_hnsw)` as a wrapper if needed (though that introduces a slight overhead). Also, because HNSW can't remove vectors, if your use case requires deletions or frequent updates, you may need to rebuild the index or use IVF indexes which do allow removals.

Binary Vector Indexes (Binary Flat and IVF)

Faiss also supports indexing of **binary vectors** (bit strings) for Hamming distance search. These are useful if your data is already in binary form (e.g. fingerprint hashes, binary descriptors). The binary indexes provide efficient Hamming distance computation.

- **Classes:** `faiss.IndexBinaryFlat`, `faiss.IndexBinaryIVF`, `faiss.IndexBinaryHash` (LSH), etc.
- `IndexBinaryFlat` stores all binary vectors and performs exhaustive Hamming distance search.
- `IndexBinaryIVF` does IVF-style clustering on binary vectors (with a binary centroid quantizer) and then Hamming distance within clusters.
- `IndexLSH` (which is actually a type of `Index`) hashes float vectors into binary codes – not to be confused with indexing pre-existing binary data (we will not focus on LSH here).

- **Binary Data Format:** The vectors should be provided as `uint8` arrays, where each vector is a sequence of bytes (each byte representing 8 bits). For example, for a 256-bit vector, use a `uint8` array of length 32 for that vector.
- **Distances:** These indexes use **Hamming distance** as the metric (count of differing bits). Faiss returns Hamming distances as integers (0 = identical, higher = more dissimilar).
- **Usage:** Using `IndexBinaryFlat` is straightforward, analogous to `IndexFlat`:

```
index_bin = faiss.IndexBinaryFlat(code_size) # code_size in *bits* (e.g.,
256 for 256-bit vectors)
index_bin.add(xb) # xb is ndarray of shape (n, code_size/8) dtype=uint8
D, I = index_bin.search(xq, k)
```

The search results `D` will be Hamming distances (int32).

- **IVF for Binary:** `IndexBinaryIVF` can be used for large sets of binary vectors. It requires training, similar to IVF for floats. Faiss will cluster the binary vectors using a **coarse binary quantizer**. Searching uses `nprobe` lists as well. The configuration is analogous: you provide a `IndexBinaryFlat` as the coarse quantizer and specify `nlist`. Example:

```
coarse_quantizer = faiss.IndexBinaryFlat(code_size)
index_bin_ivf = faiss.IndexBinaryIVF(coarse_quantizer, code_size, nlist)
index_bin_ivf.train(xb)
index_bin_ivf.add(xb)
index_bin_ivf.nprobe = 10
D, I = index_bin_ivf.search(xq, k)
```

- **Accuracy:** Binary vectors and Hamming distance have different properties. These indexes are exact for binary data (if you use `BinaryFlat`) or approximate if using IVF. Keep in mind that if your data started as float vectors that were binarized (through hashing or quantization), some information is lost, and Hamming distance might not correlate perfectly with original similarity beyond a certain point. Still, binary indexes are very fast and memory-efficient (storing 256-bit vectors requires only 32 bytes).
- **LSH (Locality Sensitive Hashing):** As an aside, Faiss provides an `IndexLSH` which hashes float vectors into binary codes and then does a Hamming distance search ²⁸ ²⁹. However, LSH is generally not very effective for high-dimensional embeddings (e.g., 128+ dims from transformers) because it requires very long codes for accuracy ³⁰. It's usually better to use the original float vectors with an ANN structure like HNSW or IVF in those cases. We mention it for completeness, but for most RAG text embeddings, LSH is **not recommended** due to the curse-of-dimensionality on required code lengths ³⁰.

Use Cases: Binary indexes are niche – use them if you *already have binary fingerprints* or need extremely fast Hamming lookups (and your similarity can be cast to bit similarity). For typical neural embeddings (which are float vectors), you wouldn't convert to binary unless you have a specific need, as it usually hurts accuracy. So for RAG (text embeddings), the focus is on the float vector indexes (Flat, IVF, HNSW, PQ, etc.) rather than binary.

ID Mapping (IndexIDMap) for Custom IDs

By default, most Faiss indexes (Flat, HNSW, PQ, etc.) assign implicit integer IDs to vectors in the order they are added (0-based). If you want to assign your own IDs to vectors – for example, document or chunk IDs in a retrieval system – you can use **ID mapping indexes**.

- **Classes:** `faiss.IndexIDMap`, `faiss.IndexIDMap2`. These are wrapper indexes that hold an internal reference to a base index.
- `IndexIDMap(index)` wraps a given index to support custom IDs. It maintains an internal mapping array from the sequential index to your provided ID ³¹ ³².
- `IndexIDMap2(index)` is similar but also supports the `reconstruct` method and may use a hash map internally (allowing non-sequential IDs efficiently). In practice, you can often use `IndexIDMap2` interchangeably with `IndexIDMap` for most Python use cases.
- **When to use:** If your base index does **not support** `add_with_ids` (most flat indexes, HNSW, etc. do not), wrapping it with IDMap enables that functionality ³. If your base index *does* support `add_with_ids` (like IVF indexes do), you don't need an IDMap – you can directly use `add_with_ids` on the IVF index. In fact, using IDMap on an IVF is redundant and wastes memory by duplicating ID storage ¹¹.
- **Usage:**

```
base_index = faiss.IndexFlatL2(d)
index = faiss.IndexIDMap(base_index)
index.add_with_ids(data_vectors, id_array)
```

Here, `id_array` is a 1D `int64` NumPy array of the same length as `data_vectors`, containing your desired IDs (e.g., document IDs). After adding, when you search the `index`, the resulting `I` array will contain your custom IDs instead of 0-based indices.

- **Important:** The IDs you provide must be **unique 64-bit integers** (Faiss uses `int64` for ids). Avoid using `-1` as an ID because Faiss reserves -1 to indicate “invalid” or “not found” in search results ³³.
- **Internals:** `IndexIDMap` simply keeps a vector of ids in memory parallel to the vectors in the base index ³⁴. So if you remove an element or reorder, it must keep them in sync. Removals are supported (IDMap's remove will remove the entry from its list) ³⁵. `IndexIDMap2` uses a hash map for mapping id to location, which can handle non-sequential ids and may be better if you plan to frequently remove or have sparse id space.
- **Index Reconstruct with IDMap2:** If you need to retrieve vectors by ID (inverse of search), note that `IndexFlat` doesn't support `index.reconstruct(id)` unless wrapped. `IndexIDMap2` specifically mentions support for `reconstruct` (so you can do `index_idmap.reconstruct(custom_id)` if the id exists). With `IndexIDMap`, you would have to map the external ID to the internal sequential index yourself (e.g., by storing your own dict from id -> internal index when adding).
- **Example usage:**

```
# Suppose we have 5 vectors and we want to assign custom IDs 10,11,12,13,14 to them.
base_index = faiss.IndexFlatIP(d)           # base index (IP metric for example)
```

```

index = faiss.IndexIDMap(base_index)           # wrap it to allow custom IDs

vectors = np.random.rand(5, d).astype('float32')
ids = np.array([10, 11, 12, 13, 14], dtype=np.int64)
index.add_with_ids(vectors, ids)

# Now perform a search
D, I = index.search(vectors[:1], k=3)
print("Neighbors IDs:", I) # will print the custom IDs (e.g., [10, 14, 12]
etc.)

```

In this snippet, we see that the search results `I` will contain our custom IDs (not 0,1,2...). The mapping ensures consistency between insertion and query.

For RAG pipelines, using an ID map is extremely useful: you can map Faiss results directly to the original text chunk IDs or indexes, which then allows you to fetch the actual text. We will demonstrate this in the RAG workflow example later.

IndexPreTransform (Preprocessing Pipelines)

Faiss allows you to compose indexes with preprocessing transforms. An `IndexPreTransform` applies one or more transformations to vectors **before** indexing or searching. This is useful for tasks like dimensionality reduction (PCA), rotation (OPQ), or normalization (L2 normalization) as part of the index.

- **Class:** `faiss.IndexPreTransform`. It wraps a base index along with one or more `VectorTransform` objects (like PCA or Normalization) that preprocess the vectors ³⁶ ³⁷.
- **VectorTransform examples:**
 - `faiss.PCAMatrix(d_in, d_out)` – a trained PCA that reduces dimensionality from `d_in` to `d_out`.
 - `faiss.OPQMatrix(M, d_in, d_out)` – optimized product quantization rotation (often used with PQ indexes to improve compression).
 - `faiss.RandomRotationMatrix(d_in, d_out)` – random projection.
 - `faiss.NormalizationTransform(d)` – L2 normalization to unit length (this uses `L2norm` under the hood) ³⁸.
 - `faiss.RemapDimensionsTransform(d_in, d_out, pad)` – pad or truncate dimensions (useful to achieve certain dimensions, e.g., make dimension a multiple of 4 or of PQ segment size) ³⁹ ⁴⁰.
- **Usage:** You can either construct an `IndexPreTransform` directly or use the convenient `faiss.index_factory` with a string description. For direct usage:

```

# Example: Normalize vectors then use FlatIP index
norm_transform = faiss.NormalizationTransform(d)
base_index = faiss.IndexFlatIP(d)
index = faiss.IndexPreTransform(norm_transform, base_index)
index.add(data) # data will be normalized before adding
# Queries will also be normalized internally before search.

```

Once wrapped, you interact with `index` as if it were a regular index (add, search, etc.). The transform is applied transparently on the fly ³⁷ ⁴¹. If the transform requires training (e.g. PCA or OPQ needs training data), calling `index.train(data)` on the `IndexPreTransform` will train both the transform and the underlying index as needed ⁴². For instance, if you have `IndexPreTransform(PCA, IndexIVFPQ)`, a single `train()` will first train the PCA (if not already trained) and then use the PCA-transformed data to train the IVF quantizer and PQ codebooks.

- **Why use PreTransform:**

- **Normalization:** Many times, you want cosine similarity. You can either normalize all vectors yourself or let Faiss do it. Using `NormalizationTransform(d)` + `IndexFlatIP` is effectively an index that searches by cosine similarity (because both stored vectors and queries are normalized automatically). This is convenient and ensures any new vector added is normalized consistently.
- **Dimensionality Reduction:** If your vector dimension is very high (e.g., 1024) but you suspect a lower-dimensional subspace is sufficient, you might apply PCA to reduce to, say, 256 dimensions before indexing. This speeds up searches and reduces memory. Faiss's `PCAMatrix` can be trained on your data for this.
- **Optimized PQ:** If using PQ, an `OPQMatrix` can significantly improve accuracy for the same code size by learning a rotation that better aligns variance with subvector blocks ⁴³.
- **Padding:** Some low-level Faiss routines are optimized when dimension is a multiple of 4 or 8 (due to SIMD alignment). Padding with zeros using `RemapDimensionsTransform` can squeeze a little more speed.
- **Index Factory:** A shorthand is to use `faiss.index_factory(dim, description_string)`. For example:

```
index = faiss.index_factory(128, "L2norm,IVF100,PQ8")
```

This would create a pipeline: normalize vectors to unit length, then an IVF index with 100 clusters and product quantization (8 bytes per vector) ⁴⁴. Another example:

```
index = faiss.index_factory(512, "PCA256,Flat")
```

would apply PCA to reduce from 512D to 256D, then use a Flat index in 256D ⁴⁴. The factory is convenient but it's essentially doing the same thing as manually composing `IndexPreTransform` and others. (Note: the index factory's syntax is documented on Faiss wiki; components are comma-separated. E.g., `"PCA80,Flat"` or `"OPQ16_64,IVF256,PQ16"` etc.)

- **Impact on Search Output:** The search results (distances) are computed in the transformed space. E.g., if you normalize and use IP, the distance returned is actually the inner product of unit vectors (which is cosine similarity). If you reduced dimensionality, distances are in the reduced space (not directly comparable to original space distances, but usually that's fine).
- **Memory Overhead:** The transforms themselves take some memory (to store matrices, etc.) but usually negligible compared to the index. The big consideration is that if you use `IndexPreTransform` to reduce dimension, you might be saving memory in the index. If using it to normalize or rotate, memory usage is roughly unchanged (just storing the same vectors, possibly transformed).

Example – Index with L2 Normalization Preprocessing

Below we combine normalization with an inner-product flat index, which effectively creates a cosine-similarity index:

```
d = 128
data = np.random.rand(1000, d).astype('float32')
# Create Normalization + FlatIP pipeline
norm = faiss.NormalizationTransform(d)      # L2 normalize vectors
index_cosine = faiss.IndexPreTransform(norm, faiss.IndexFlatIP(d))
index_cosine.add(data)

# Verify that stored vectors are unit norm (just to illustrate):
vec = index_cosine.reconstruct(0)           # reconstruct first vector
print("Norm of first stored vector:", np.linalg.norm(vec))

# Search (the search will also normalize the query internally)
query = np.random.rand(1, d).astype('float32')
D, I = index_cosine.search(query, k=5)
print("Cosine index top-5 scores:", D)
print("Top-5 indices:", I)
```

In this code, `index_cosine` will automatically normalize any vector added or queried. We even call `reconstruct` (supported because `IndexPreTransform` passes this through to the base index, applying inverse of transform if possible) to see that the stored vector has unit norm. The search distances `D` here are actually **cosine similarities** (range -1 to 1) because of the normalization + inner product combination. A higher score means more similar.

IndexRefineFlat (Re-ranking with Exact Search)

`IndexRefineFlat` is an index that wraps another base index to **refine its results with a second-stage exact search** ⁴⁵ ⁴⁶. The typical use case is: you have an approximate index (like IVFPQ or HNSW) that returns *candidate neighbors*, and you then re-rank those candidates by computing the true distances using the original vectors (stored in a flat index) to improve accuracy.

- **Class:** `faiss.IndexRefineFlat(base_index)`. The `base_index` is your first ANN index. The `IndexRefineFlat` will store a copy of all vectors in an internal `IndexFlat` (this doubles memory, since you keep both the base index's data and a full copy for refinement).
- **k_factor:** A parameter `refine_index.k_factor` determines how many results the base index should return internally for refinement. By default, when you call `search(nq, k)`, `IndexRefineFlat` will ask the base index for `k * k_factor` neighbors, then re-compute their true distances and select the top-k ⁴⁵ ⁴⁶. If `k_factor=1`, then `IndexRefineFlat` does nothing special (just uses the base results as is). If `k_factor=2`, and you ask for 10 neighbors, it will get 20 from base, then re-rank and return the best 10 of those 20 by exact distance.
- **Training and Adding:** Typically, you build and train the base index, then use `IndexRefineFlat`. You need to add the same vectors to the `IndexRefineFlat` as well. Actually, when you construct it as `refine_index = faiss.IndexRefineFlat(base_index)`, Faiss provides a convenient constructor that can optionally take the whole dataset to add to the refinement index immediately

⁴⁷ ⁴⁸ . If not, you must call `refine_index.add(x)` with your dataset (which will actually add to the internal Flat index; the base index should already have those vectors).

- **Search:** Querying a RefineFlat will perform: `base_index.search` (with `k*k_factor`) -> get intermediate I, then do exact distance computation for those candidates -> final re-ranking. This typically improves recall/precision especially if your base index compresses vectors (PQ) or uses approximate distances. It can substantially boost accuracy at the cost of some additional compute for the refinement stage.
- **Pros & Cons:** Pro – better accuracy without too much extra work, especially if base index had lossy compression. Con – doubles memory (needs a full copy of vectors in the refine index) and increases query time (because of the second stage). It's essentially implementing a *hybrid two-stage retrieval*: fast but approximate first stage, then exact re-score of a shortlist.
- **Use Case:** If you use heavy compression (like IVFPQ with very short codes), an IndexRefineFlat can recover a lot of the lost accuracy. In RAG contexts, if you absolutely need the most relevant passages and can afford a bit more latency, refine-flat is an option. However, many times you can also mitigate by just using a less aggressive compression or higher `nprobe` instead. RefineFlat is most beneficial when you want to keep the base index very fast (low accuracy) but then correct it. For example, `IVFPQ (8-byte codes)` with `IndexRefineFlat` often can reach accuracy close to an uncompressed index but still store vectors in 8 bytes for the coarse stage ⁴⁶ .
- **Example:**

```
base_index = faiss.IndexPQ(d, 16, 8)           # 16x8=128-bit PQ (requires
base_index.train(xb)                           training)
base_index.add(xb)
refine_index = faiss.IndexRefineFlat(base_index) # make refine index over
base
refine_index.k_factor = 4                      # e.g., retrieve 4x
candidates
D, I = refine_index.search(xq, k=10)
```

Here, `base_index` might be very lossy (PQ compresses to 16 bytes per vector). We set `k_factor=4` so that if we want 10 final neighbors, it fetches 40 from the PQ and re-ranks. The `IndexRefineFlat` internally stored a full copy of `xb` in a flat index (so memory = original dataset * sizeof(vector) + PQ index memory).

IndexShards and IndexReplicas (Combining Multiple Indexes)

Faiss provides composite indexes to aggregate multiple indexes for either **sharding** or **replication**:

- **IndexShards (Sharded Index):** This index is used to distribute a dataset across multiple sub-indexes (shards). When you add vectors to an `IndexShards`, it will add them to the shards in sequence or in a round-robin fashion. When you search, it queries all shards and merges the results. This is useful if you want to split data across multiple machines or processes (or GPUs) but still query as one index ⁴⁹ ⁵⁰ . For CPU usage, you might use it to split data into chunks and use multi-threading to search each chunk in parallel.
- You can create it via `faiss.IndexShards(dim, threaded=True, successive_ids=True)`. Then you must `index_shards.add_shard(subindex)` for each shard index you have prepared. If `threaded=True`, Faiss will search shards in parallel threads.

- `successive_ids=True` means if you don't provide IDs, it will assign IDs that continue across shards (so shard0 will use 0..n0-1, shard1 uses n0..n0+n1-1, etc.).
- When searching, it collects results from all shards and keeps the top-K overall.
- **Use Case:** Large dataset that doesn't fit in memory of one machine, or wanting to speed up search by dividing work. In a pure Python context, you might not use this heavily unless you manage multiple processes. But for multi-GPU it's definitely used (Faiss uses `IndexShards` under the hood for multi-GPU indices with `index_cpu_to_gpu_multiple`).
- On CPU, if you have a multicore machine, Faiss already uses multithreading within a single index (especially Flat and IVF), so `IndexShards` is more relevant if you manually partition data or use separate machines.
- **IndexReplicas (Replicated Index):** This is conceptually the opposite of sharding: each sub-index is a complete copy of the dataset, and a query is dispatched to all and results merged. This can be useful to increase throughput (each replica can handle queries in parallel), or to aggregate different modalities of search (though for the latter Faiss also has other tools).
- In Faiss C++ API, `IndexReplicas` exists for multi-GPU to split a large batch of queries across GPUs (each GPU has a copy of the whole index, handling some queries).
- In CPU context, you might rarely use it unless you want to utilize multiple threads by replicating the index in multiple threads (which is memory-inefficient). Most users stick to Faiss's internal threading or sharding rather than replication on CPU.

In summary, `IndexShards` is for horizontally partitioning the data, `IndexReplicas` for vertically partitioning the queries or providing redundancy. Both can be constructed and used in Python, but they are advanced and typically used when you have special needs (like datasets exceeding one machine's RAM, or combining indexes built on different subsets). For completeness, this guide acknowledges them, but many RAG scenarios won't require manual shards/replicas unless scaling out.

Range Search (Finding Neighbors within a Radius)

Besides the standard k-NN search, Faiss supports **range search**: retrieving all vectors within a given distance radius of the query. This can be useful for queries like "find all vectors with distance < τ ". On CPU, range search is supported for a subset of indexes (Flat, IVFFlat, IVFSQ, etc.) ⁵¹.

- **Usage:** `lims, D, I = index.range_search(xq, radius)`. Here:
- `xq` is query vectors (shape `(nq, d)`),
- `radius` is the radius threshold.
- The result is given in three arrays:
 - `lims`: size `nq+1`, where `lims[i]` and `lims[i+1]` demarcate the segment of results for query `i` in the `D` and `I` arrays.
 - `D`: distances of all found neighbors concatenated.
 - `I`: labels (ids) of all found neighbors.
- For each query `i`, the indices of its neighbors are `I[lims[i]:lims[i+1]]` and corresponding distances `D[lims[i]:lims[i+1]]` ⁵² ⁵¹.

- **Example:** If you have one query and `lims = [0, r]` where `r` is the number of results found, then `I[:r]` are the neighbors. If you have two queries and `lims = [0, r1, r1+r2]`, then results for query0 are `I[0:r1]` and for query1 are `I[r1:r1+r2]`.
- **Performance:** Range search can be more expensive than k-NN if many points fall within the radius. Internally, Faiss will still traverse the index structures, but it can't stop early after k results, it must gather all within threshold. Use it judiciously – for sparse results or for evaluation purposes. For dense datasets, a large radius might return huge result sets (and be slow).
- **Support:** The Faiss documentation notes range search is supported for flat indexes and some IVF indexes (IVFFlat, IVFScalarQuantizer) on CPU ⁵¹. It may not be implemented for all index types or on GPU (GPU had limitations historically).
- **Use Case:** For RAG, range search is not commonly used – typically you want the top-k results. But one could imagine using range search to see if a query has *any* vector above a similarity threshold (for example), or to do clustering by finding all points within a radius. Usually, k-NN is sufficient.

Example:

```
# Range search example on a flat index
index_flat = faiss.IndexFlatL2(d)
index_flat.add(data)
radius = 1.0 # e.g., find all vectors within L2 distance 1.0
lims, D, I = index_flat.range_search(query, radius)
for qi in range(query.shape[0]):
    start = lims[qi]
    end = lims[qi+1]
    neighbors = I[start:end]
    print(f"Query {qi} has {len(neighbors)} neighbors within radius {radius}: {neighbors}")
```

This will print for each query how many neighbors (and their IDs) are within the specified radius.

Hyperparameter Tuning with ParameterSpace

Faiss offers a utility class `faiss.ParameterSpace` to manage and auto-tune index parameters. It is especially helpful to adjust search parameters (like `nprobe`, `efSearch`, etc.) either globally or per query.

- **Setting parameters globally:** You can directly set attributes like `index.nprobe = 8` or `index.hnsw.efSearch = 32` for many indexes. However, some parameters are nested or require consistent setting (e.g., an IVF's quantizer might be HNSW and needs `quantizer_efSearch`). `ParameterSpace` provides a unified interface:
- Example: `faiss.ParameterSpace().set_index_parameters(index, "nprobe=5, quantizer_efSearch=8")` will find the relevant parameters in the index and set them ⁵³. This is useful if you have an index that contains sub-indexes (like IVF with HNSW quantizer) – as shown, you prepend `quantizer_` to target the coarse index's parameters ⁵⁴ ⁵⁵.
- This method throws an error if a parameter is not applicable to that index, helping catch mistakes.
- **Per-query parameters:** Instead of changing the index state, you can supply a `SearchParameters` object when calling `search`. Faiss defines subclasses like `SearchParametersIVF`,

`SearchParametersIVFPQ`, `SearchParametersHNSW`, etc., that correspond to the index type ⁵⁶
⁵⁷. For example, for an IVFPQ index:

```
params = faiss.SearchParametersIVFPQ(nprobe=10)
D, I = index.search(xq, k, params=params)
```

This will use `nprobe=10` for that search call only, leaving the index's default `nprobe` unchanged ⁵⁸. This is thread-safe, as different threads can supply different params.

- Another example: if the IVF's coarse quantizer is HNSW, you can nest parameters:

```
params = faiss.SearchParametersIVF(nprobe=5)
params.quantizer_params = faiss.SearchParametersHNSW(efSearch=64)
D, I = index.search(xq, k, params=params)
```

This would set IVF `nprobe=5` and the HNSW quantizer's `efSearch=64` just for this query ⁵⁹.

- For HNSW standalone index, you might have `params = faiss.SearchParametersHNSW(efSearch=50)`.
- **Auto-tuning:** Faiss's `ParameterSpace` can also search for optimal combinations of parameters if you provide ground truth data (using `AutoTuneCriterion` and `parameter_space.explore()`), which yields an `OperatingPoints` object of speed/accuracy trade-offs ⁶⁰ ⁶¹. This is beyond the scope of this guide, but it's good to know if you have a benchmark dataset to optimize on, Faiss can try various `nprobe/efSearch` etc. and suggest Pareto-optimal settings.
- **Common tunables:**
 - IVF: `nprobe`
 - IVF with multi-index (IMI): `max_codes`
 - HNSW: `efSearch`
 - PQ (standalone or within IVF): `ht` (Hamming threshold for polysemous codes, an advanced topic for byte-level refinement – often not used) ⁶².
 - Refine (IndexRefineFlat): `k_factor` (not sure if `ParameterSpace` covers that, but it's an attribute you can set).
- **When to use `ParameterSpace` vs direct:** For simple cases, setting the attribute is fine (`index.nprobe = 10`). Use `SearchParameters` if you need to vary per query or avoid thread interference. Use `ParameterSpace` if you want to programmatically explore or apply a set of parameters without writing index-type-specific code paths.

Removing Vectors (Deletion) in Faiss Indexes

Deleting vectors in Faiss must be done with care, as not all index types support it easily:

- **Supported Indexes:** According to Faiss documentation, **Flat indexes**, **IVF indexes (with some conditions)**, and **IDMap** support removal via the `remove_ids` method ⁶³ ³⁵. HNSW does *not* support removals ²⁶, and neither do PQ or LSH flat indexes in a straightforward way (if they are standalone sequential indexes, removal will shift IDs).

- **Method:** `index.remove_ids(selector)` removes all vectors that satisfy an ID selector. The simplest way in Python is to pass a NumPy array of ids to remove – Faiss will construct an `IDSelectorBatch` internally for those IDs ⁶³ ⁶⁴. For example:

```
to_remove = np.array([42, 123, 7], dtype=np.int64)
index.remove_ids(to_remove)
```

This will remove vectors with those ids (if they exist). Under the hood, an `IDSelectorBatch` is created with that id list ⁶³.

- **Cost:** Removal is not necessarily cheap. The `remove_ids` operation may scan the whole index to find the IDs to remove ⁶⁴, especially for flat indexes (O(N) operation). It's efficient only if you remove a significant fraction of the dataset; removing single or very few points repeatedly is not optimal.
- **ID impacts:**
 - For **Flat (sequential) indexes:** Removing an entry will cause all higher-index entries to shift down by one (their id -1) ⁶. This means if you stored external IDs via `IndexIDMap`, those external IDs remain attached to the vectors themselves, so from a user perspective, if you always use custom IDs, you're fine. But if you were relying on position, positions change.
 - For **IVF indexes:** Since IVF stores explicit vector IDs in the inverted lists, removing vectors does not affect the IDs of other vectors ⁶⁵ (each vector keeps its id in the list). However, if the IVF was using `DirectMap` of type `Array` (which assumes sequential IDs), that mode doesn't support removal because the array of IDs would need shifting ⁶⁶. The default `DirectMap.NoMap` or using `DirectMap.Hashtable` (enabled by `index.set_direct_map_type(faiss.DirectMap.Hashtable)`) is needed for IVF removal to be efficient.
 - For **IndexIDMap2:** It also stores explicit IDs, so removing doesn't shift others (similar reasoning as IVF) ⁶⁵.
- **Example:**

```
index = faiss.IndexIDMap(faiss.IndexFlatL2(d))
ids = np.arange(1000, dtype=np.int64)
index.add_with_ids(data, ids)
# Remove ids 100, 200, 300
index.remove_ids(np.array([100, 200, 300], dtype=np.int64))
```

After this, the index's `ntotal` will have decreased by 3. A search for those IDs will yield -1 (as removed) if attempted.

- **Alternatives:** If you need frequent dynamic updates, a brute-force approach is to keep an external set of “active” IDs and filter search results instead of removing (see filtered search next). Or consider a different data structure. Faiss isn't a database; heavy dynamic updates are not its primary design (though it can handle some).
- **Sanity on HNSW:** If you absolutely must remove from HNSW, you effectively have to rebuild or maintain your own tombstone list to ignore certain IDs during search (again, see filtered search below).

Filtered Search (Searching a Subset of Vectors)

Sometimes you want to query only a subset of the indexed vectors (for example, restrict to a certain user's data, or filter out deleted items without actually removing them). Faiss allows *filtered search* using **IDSelector** objects in combination with **SearchParameters**.

- **IDSelector:** This is a class (in C++ and Python) that defines a set of allowed IDs. Faiss provides several implementations:
 - `faiss.IDSelectorBatch(ids)` – selects a given list of ids (implemented with a hash set and bloom filter for speed) ⁶⁷ ⁶⁸ .
 - `faiss.IDSelectorArray(ids)` – also selects a list, but implemented as a simple array check (not as efficient for large sets) ⁶⁹ .
 - `faiss.IDSelectorBitmap(bitmap)` – selects all ids present in a bitmap array (useful if a large contiguous range is selected) ⁷⁰ ⁶⁸ .
 - `faiss.IDSelectorRange(imin, imax)` – selects ids in a given range [imin, imax) ⁷¹ .
 - `faiss.IDSelectorNot(sel)` – negates another selector (selects ids that are *not* in the given selector set) ⁷² .
 - `faiss.PyCallbackIDSelector(func)` – wraps a Python function `func(id) -> bool` to decide membership (convenient but very slow, as it calls Python for every vector) ⁷³ ⁷⁴ .
- **Using selectors in search:** Normally, you pass a **SearchParameters** object to `index.search()`. These objects (like **SearchParametersIVF**, etc.) have a field `.sel` for an IDSelector ⁷⁵ . So:

```
selector = faiss.IDSelectorBatch(np.array([1,2,3], dtype=np.int64))
params = faiss.SearchParametersIVF(nprobe=5)
params.sel = selector
D, I = index.search(xq, k, params=params)
```

This would make the search consider only vectors with IDs in {1,2,3} as possible results ⁷⁵ . In this trivial example, there are only 3 allowed vectors, but imagine a larger subset.

- **When to use:** Filtered search is useful for multi-tenant scenarios (search only within a user's subset), or enforcing some constraint without rebuilding separate indexes. It can also be used to *exclude* some IDs (e.g., IDSelectorNot with a batch) – for example, to *ignore deleted vectors* without actually removing them (just mark them and filter out).
- **Performance considerations:** Filtering does add overhead – each candidate must pass through the `is_member` check of the selector. For large allowed sets, IDSelectorBatch is optimized with a hash set and even a bloom filter for large n ⁶⁸ . For range selections on sorted IDs (like if your index's IDs are roughly ordered), there are optimizations in IVF to skip entire inverted list sections ⁷⁶ . In the worst case (if your filter is almost as large as the dataset), it will degrade performance close to an unfiltered search plus checking each id.
- **Example (Filtering by IDs):**

```
# Suppose we only want to search within ids [1000, 2000)
sel = faiss.IDSelectorRange(1000, 2000)
params = faiss.SearchParametersIVF(nprobe=5)
params.sel = sel
```

```
D, I = index_ivf.search(xq, k, params=params)
# Now I will only contain ids in [1000,2000) (assuming those exist in the index)
```

Here we used `IDSelectorRange` to only search a continuous range of ids. Faiss can optimize this for IVF if IDs in the inverted lists are stored sorted (which they typically are if inserted in order) ⁷⁶.

Note: When using ID maps, the “id” in Faiss’s terms is the one you added (e.g., your external ID). So selectors would operate on those same IDs. If you used `IndexIDMap` with custom IDs, then filtering by ID will directly use those custom IDs.

For RAG use cases, filtered search might be less common, but imagine you had documents tagged by category and you wanted to restrict retrieval to a certain category – if your ID scheme encodes category or you maintain a mapping from category to ids, you could pre-filter by that subset.

With the above building blocks covered, we can now move to how to adapt GPU workflows to CPU, and then concrete examples for RAG pipelines and best practices.

Converting GPU-Based Faiss Workflows to CPU

If you have been using Faiss on GPU (with classes like `GpuIndexFlat` etc. from `faiss-gpu` or a custom GPU build like a cuVS-enabled version), here’s how to transition those workflows to CPU equivalents. The good news is that **Faiss API is very similar between GPU and CPU**, so in many cases it’s replacing `GpuIndex*` with the corresponding CPU index class and removing GPU-specific resource management.

Below are common GPU patterns and their CPU counterparts:

- **Standard GPU Index Classes:**

- `faiss.GpuIndexFlatL2` or `GpuIndexFlatIP` → Use `faiss.IndexFlatL2` / `IndexFlatIP` on CPU. The usage of adding and searching is identical. On GPU, you typically had to create a `StandardGpuResources()` and maybe use `faiss.index_cpu_to_gpu`. On CPU, you simply instantiate the index directly. For example, instead of:

```
res = faiss.StandardGpuResources()
index = faiss.GpuIndexFlatIP(res, dim)
```

on CPU you do:

```
index = faiss.IndexFlatIP(dim)
```

There is no `res` needed, and no GPU device specification. All data stays in main memory.

- `faiss.GpuIndexIVFPQ` → **Use** `faiss.IndexIVFPQ`. The same parameters (`nlist`, `m`, `nbits`) are used. On GPU, you might have built a CPU index and then transferred with `index_cpu_to_gpu`; on CPU you just use it directly. One difference: on GPU you might use `float16` or `indices` options via `GpuIndexIVFPQ` (like configuring it to use FP16 storage for centroids etc.). On CPU, the implementation uses `float32` always for calculations (unless you use quantization like PQ which is by definition compressed).
- `faiss.GpuIndexIVFFlat` → **Use** `faiss.IndexIVFFlat` correspondingly. If you were using `GpuIndexIVFFlat` for brute-force on GPU with clustering, the CPU `IndexIVFFlat` will behave the same but run on CPU threads.
- `faiss.GpuIndexHNSW` (if exists with `cuVS`) → **Use** `faiss.IndexHNSWFlat`. As of Faiss 1.7/1.8, the HNSW implementation on GPU (via `cuVS`) could build an HNSW graph on GPU (called `GpuIndexHNSW` or `GpuIndexCagra` in some contexts) ⁷⁷. On CPU, you'll use the standard HNSW as discussed. Note: If your GPU workflow used `GpuIndexCagra` (the NVIDIA accelerated graph index builder), the closest CPU analogue is an `IndexHNSWFlat` (Cagra builds an HNSW index). You may have even used `index_cpu_to_gpu(res, 0, cpu_hnsw_index, GpuClonerOptions(use_cagra=True))` in a GPU build – on CPU you just use that HNSW index directly (but expect much slower build times without GPU acceleration).
- **GpuIndexShards / Multiple GPUs:** If you used `faiss.IndexShards` or `index_cpu_to_gpu_multiple` to shard across GPUs, on CPU you could either combine shards via `IndexShards` (if you want to use multiple processes or machines), or if everything fits in one CPU, simply concatenate data into one index. CPU multi-threading can often handle a lot, but if you truly need distribution, consider using `IndexShards` across workers (though that's beyond a single process – you'd likely implement a fan-out of queries).

• Gpu Cloning and Resources:

- On GPU, workflows often do:

```
res = faiss.StandardGpuResources()
index_gpu = faiss.index_cpu_to_gpu(res, device, index_cpu, gpu_options)
```

Or use `faiss.GpuClonerOptions` for advanced control (like `useFloat16=True`, `devices=[0,1]`, etc.). On CPU, none of this is needed. If you have a saved index from GPU, you can simply load it with `faiss.read_index` on CPU (Faiss indexes are portable between CPU/GPU for most types). If the index was built on GPU with GPU-specific parameters (e.g., a certain layout), Faiss should still read it on CPU transparently (with maybe some exceptions for very GPU-specific constructs).

- The `GpuClonerOptions.useFloat16` would have stored vectors in half precision on GPU. On CPU, they will be converted back to `float32` in memory. If memory is a concern, you might consider using an `IndexScalarQuantizer` on CPU to simulate a similar memory compression.

- The `GpuClonerOptions` also had `shard` or `use_precomputed` options. Those have no equivalent on CPU because CPU doesn't precompute LUTs in the same way (except PQ precomputed tables which are always used on CPU).

- **Brute-force kNN on GPU (`faiss.knn_gpu` / `bfKnn`):**

- If you were using `faiss.knn_gpu(res, queries, database, k, ...)`, the CPU equivalent is `faiss.knn` (for brute-force on CPU) or just create an `IndexFlat` and call `search`. The `faiss.knn` function in Faiss can compute a brute-force kNN between two sets of vectors using CPU (possibly leveraging BLAS). For example:

```
distances, indices = faiss.knn(queries, database, k)
```

This requires both `queries` and `database` to be numpy float32 arrays. It returns the same result as if you did:

```
index = faiss.IndexFlatL2(d)
index.add(database)
distances, indices = index.search(queries, k)
index.reset()
```

The `faiss.knn` is basically a convenience that avoids explicitly creating an index (and might optimize by choosing BLAS or other paths behind the scenes). For clarity in code, many stick to making an `IndexFlat`.

- If you used `bfKnn` (perhaps an alias or custom for brute-force), treat it the same as above.

- **GPU-specific utilities:**

- `faiss.StandardGpuResources` – no need on CPU.
- `faiss.pin_memory` or memory transfer calls – not relevant on CPU.
- GPU memory concerns: On CPU you typically have more memory (and virtual memory swapping if needed), so indices that didn't fit on GPU might fit on CPU. However, CPU memory bandwidth is lower, so large brute-force searches could be slower.
- If your GPU workflow was using the **RAFT (Rapid AI Faiss) integration** (like `useRAFT=True` on `knn_gpu` or special RAFT indexes), note that those are specific to GPU and have no direct CPU counterpart. The CPU always uses its built-in algorithms. You might see differences in performance: for instance, RAFT optimized brute-force might have been faster for certain batch sizes on GPU; on CPU, multi-threaded BLAS will be used.

- **Performance differences:**

- GPU excels at large, parallel computations. CPU Faiss uses multi-threading (OpenMP) to parallelize over queries or data where possible ⁷⁸ ⁷⁹. If you move from GPU to CPU, expect that for very large datasets or high-dimensional data, queries will be slower – possibly by an order of magnitude or

more – unless you compensate by using more approximation (e.g., use IVF with fewer computations).

- Memory bandwidth on CPU is lower, which affects brute force searching especially. For example, a V100 GPU might scan vectors in memory much faster than a CPU can. Therefore, consider using approximate methods (IVF, HNSW) on CPU where you might have done brute force on GPU.
- Batch queries: On GPU, it's common to batch many queries for efficiency. On CPU, batching helps too (Faiss will parallelize over queries by default), but extremely large batch sizes might not scale the same way. It's usually fine and recommended to submit queries in batches rather than one by one to utilize vectorized instructions and cache.
- Index build time: Clustering (training IVF, PQ) on CPU uses k-means which is multi-threaded but could be slow for huge data (GPU could do it faster if using `GpuIndexIVFFlat` training on GPU). For millions of vectors, consider sampling the training data if needed. The result might be slightly less optimal but faster to train.
- HNSW build: If you used GPU CAGRA to build HNSW quickly, on CPU building HNSW for millions of vectors is slow. You might either allocate more threads (`omp_set_num_threads`) or accept the longer build. You could also consider building on GPU then transferring to CPU: Faiss supports copying an index from GPU to CPU (`index_gpu_to_cpu`). For instance, you can use a GPU to build HNSW (with cuVS) and then do `faiss.index_gpu_to_cpu(gpu_index)` to get a CPU index populated ⁸⁰. Since the guide is focusing on CPU usage, we assume you may not want to involve GPU at all, but it's an option if available.
- **GpuParameterSpace:** On GPU, there's `faiss.GpuParameterSpace` for tuning GPU indices (including some GPU-only parameters). On CPU, use `faiss.ParameterSpace` as described. Most of the important tunables (nprobe, efSearch) are shared. GPU had additional things like deciding whether to use CPU or GPU for certain parts (not applicable here).

Summary: To convert GPU code to CPU: - Drop all GPU resource initialization. - Use CPU index classes (`IndexFlat*`, `IndexIVF*`, `IndexHNSW*`, etc.). - Remove or replace GPU-specific function calls (like `index_cpu_to_gpu` -> just use the index, `knn_gpu` -> use `knn` or `Index.search`). - Adjust for performance by possibly choosing different index types since you can't rely on massive GPU parallelism. E.g., if you did brute force on GPU for 1M vectors, on CPU you likely need an IVF or HNSW to get acceptable speed. - Ensure data is in numpy float32 and contiguous before calling Faiss on CPU (this was also necessary for GPU in many cases). - Multi-threading on CPU is automatic; if needed, you can set `faiss.omp_set_num_threads(n)` to control it (by default it will use all cores).

By following these steps, you should get equivalent functionality. Just be mindful of the speed and memory differences, and use the best practices for CPU (which we'll cover next, especially in the context of RAG).

Design Decisions for RAG (Retrieval-Augmented Generation) Systems

In retrieval-augmented generation, we typically embed a corpus of text (documents split into chunks or passages) into high-dimensional vectors, build an index for these vectors, and at query time embed the question and find relevant passages via nearest neighbor search. The scale of data in RAG can vary widely – from a few thousand FAQs to a million Wikipedia passages – but is often in the **small-to-medium** range (not

billions of vectors). This means we have flexibility in choosing index types; we want a balance of *fast query* and *high recall* to retrieve the most relevant chunks for generation.

Key considerations for RAG indexes: - **Accuracy vs Speed:** In a QA or support chatbot scenario, failing to retrieve the relevant text can hurt the answer. So recall is important. Often we prefer methods with very high accuracy (Flat or HNSW, or IVF with high nprobe) at the cost of some extra milliseconds, because generating a response (with an LLM) might anyway take more time. That said, a well-tuned IVF or HNSW can achieve ~95-99% of the recall of brute force with much less latency. - **Dataset Size:** This often dictates the index: - *Few thousand vectors:* You can comfortably use `IndexFlatIP` (if using cosine similarity) and get millisecond queries. The entire operation is negligible compared to the embedding or generation time. Use flat for simplicity and exactness. - *Tens of thousands (say 50k):* Flat is still okay (50k * 384-dim vectors – a typical embedding – is manageable). HNSW could be used if you need sub-millisecond retrieval, but often not necessary. IVF doesn't shine much here unless you really need to prune computations. - *Hundreds of thousands:* Now Flat might start to be slow (100k–500k vectors, searching all could be tens of milliseconds per query). HNSW is a strong candidate – it can handle this scale with proper memory. IVF can also be used (e.g., IVF with 1k or 5k clusters, nprobe ~10) to get speedup. Which to choose can depend on memory vs performance: - HNSW will use more memory (it stores edges) but likely give higher recall out-of-the-box. - IVFFlat or IVFPQ will use less memory (especially if PQ compresses vectors), but might need careful tuning to reach the recall you want. - *Millions of vectors:* At this scale, you likely need approximation. Flat search on 1M of 768-dim vectors might take on the order of 100-200ms single-threaded; with multi-threading perhaps ~10-30ms (depending on CPU). It might be borderline acceptable, but if you expect concurrent queries or need faster, use IVF or HNSW. HNSW with M=32 on 1M can still be fine if you have enough RAM (1M * (vector_size + graphLinks) maybe ~ 1M * (7684 bytes + 328 bytes) ≈ 3GB, not trivial but possible). IVFPQ could reduce memory to perhaps 0.5GB with 8-byte codes, but then you deal with PQ approximation. - **Memory Constraints:** Often RAG corpora are not so huge that memory is the first concern. But if you're tight on RAM (say deploying on limited hardware) or if your corpus grows to many millions: - Use **Product Quantization (PQ)** to compress vectors. E.g., storing 384-dim embeddings in 8 bytes (with IVFPQ) is a 48x compression. This might drop accuracy a bit, but if you refine or use a good k-factor, it can still work. - Use **OPQ** transformation before PQ to mitigate quantization loss (Faiss can do OPQ as a pre-transform). - Or use **IVF with scalar quantization** (that gives ~4x compression with 8-bit, lower if 4-bit). - Another idea: **Dimension reduction** via PCA. Perhaps reduce 768-dim to 256-dim with little loss in semantic accuracy (depending on your data). - If memory is extremely constrained, one could consider binary hashing (LSH or custom) to drastically downsize, but as mentioned, quality might degrade for high-dimensional semantic embeddings ²⁹. - **Which metric to use:** Most text embeddings (Sentence Transformers, etc.) are compared via cosine similarity. In Faiss, you'd either: - Normalize all embeddings and use `IndexFlatIP` or `IndexIVF..., METRIC_INNER_PRODUCT`. - Or use `IndexFlatL2` without normalization – sometimes people do this because minimizing L2 between normalized vectors is equivalent to maximizing cosine similarity (since $\|a - b\|^2 = 2 - 2 * \cos(a, b)$ when a and b are unit vectors). Either way works. A common approach is to do **on-the-fly normalization**: e.g., using `IndexFlatIP` and calling `faiss.normalize_L2(x)` on each batch of vectors (for both data and queries). - We recommend using inner product (IP) in Faiss for RAG, because you can integrate it with Faiss's transforms easily (and Faiss has some optimized inner-product search paths too). - **Index choice summary for RAG:** - For **small** (up to ~50k): **FlatIP + IDMap** is straightforward. If you need faster and can spare memory, **HNSW** is an option, but ensure no critical need for deletes or add_with_ids (or wrap with IDMap for the latter). - For **medium (50k-1M)**: **HNSW** is a good default for high accuracy. If memory usage is a concern (like >1M at 768-dim ~ a few GB), or if you want more control, use **IVFFlat** or **IVFPQ**. For IVFFlat, choose nlist such that each list has maybe a few hundred vectors (so nlist ~ N/200 to N/1000). Use `nprobe` enough to cover maybe 5-10% of

the dataset in worst case, or tune empirically. This can give you 5-10x speedups over Flat with minimal recall loss if done right. - For **larger** (several million): A combination of **IVF + PQ** might be needed to keep memory manageable. Also consider indexing on disk solutions (not covered by faiss-cpu directly unless you memory-map index files). - One might also consider **hybrid**: e.g., use a smaller vector model (like 256-dim embeddings instead of 768) to reduce memory and use Flat or HNSW on those – sometimes model choice can help more than index choice. - **Example scenario**: Suppose you have 100k FAQ entries (each a few sentences). 100k vectors of dimension 384 is ~0.15 GB in memory – trivial for most servers. Query per second might be a few, latency budget 50ms. A Flat index might already answer in ~10ms on CPU (with multithreading) – so you might not need complexity. If QPS is higher, or you deploy on weaker hardware, HNSW with efSearch tuned to get ~95% recall might cut that to 2ms queries. - **Batching queries**: If you expect to do multiple queries in parallel (like user asks multiple questions or one question is broken into sub-questions), Faiss can handle a batch easily: just stack the query vectors and call `search()`. It will use threads to process them in parallel. This is typically more efficient than looping query by query in Python due to reduced overhead ⁷⁸ ⁷⁹. - **Thread control**: By default Faiss CPU uses OpenMP to utilize all cores for many operations (unless compiled otherwise). For example, adding a large set of vectors, training k-means, or searching many vectors will spawn threads. If you need to restrict this (maybe to not use all cores), you can call `faiss.omp_set_num_threads(n)` at runtime to limit the number of threads Faiss will use internally. This can be useful in a web server environment to avoid each request using all cores. - **Similarity Score Normalization**: Note that Faiss returns either distances (for L2) or inner products. In RAG, you might want to convert these to more interpretable scores (like similarity percentages) or at least be aware how to use them. For cosine similarity, if vectors are unit norm, an inner product of 0.8 is a pretty high similarity. You might threshold or weight accordingly when deciding if a result is relevant enough to use in generation.

Selecting Top-k in RAG: Usually we fetch top-k (like k=5) passages to feed into the generator. This k is small (maybe 3 to 10). All Faiss indexes can easily return top-5 or top-10 as requested. If your index is approximate, ensure that its parameters are tuned such that it's very likely those top-5 include the actual relevant passages. It can be acceptable if rank 1 is wrong occasionally as long as the truly relevant passage is somewhere in top-5, because the generator might still incorporate it. But if the relevant info is missing entirely from the retrieved set, the answer will suffer. So aim for high recall@k.

Finally, let's consolidate everything with a practical example of building a text-vector index and using it in a RAG pipeline.

End-to-End Example: Text Embedding and RAG Query Workflow

In this section, we'll walk through a simplified example of indexing text chunks with Faiss-CPU and then using that index to answer a query.

Step 1: Embed Text Chunks – In a real system, you would use a transformer model to get embeddings for your text. Here we'll simulate embeddings or use a simple model if available.

```
# Suppose we have a list of text documents or passages
documents = [
    "What is the capital of France? The capital of France is Paris.",
    "Paris is known for the Eiffel Tower.",
```

```

    "London is the capital of the UK.",
    "The Eiffel Tower is located in Paris, France.",
    "Berlin is the capital of Germany."
]

# Step 1: Obtain embeddings for each document.
# For demonstration, we'll use a random vector for each, but in practice use a
# model.
d = 384 # embedding dimension (typical for many sentence transformers)
import numpy as np
np.random.seed(0)
embeddings = np.random.random((len(documents), d)).astype('float32')

# Optionally, normalize embeddings if using cosine similarity
faiss.normalize_L2(embeddings)

```

(In practice, you might use a library like *SentenceTransformers*: e.g., `model = SentenceTransformer('all-MiniLM-L6-v2')` then `embeddings = model.encode(documents, normalize_embeddings=True)`. Ensure the embeddings are float32 numpy arrays.)

Step 2: Build a FAISS index for these embeddings. We will use an `IndexFlatIP` wrapped in `IndexIDMap` so we can retrieve the original text by ID. Since our dataset is small (5 docs), a flat index is fine. For a larger dataset, you might choose IVF or HNSW accordingly.

```

import faiss
# Step 2: Create index (Flat IP with ID mapping)
base_index = faiss.IndexFlatIP(d) # inner product index
index = faiss.IndexIDMap(base_index) # wrap with ID map to store custom IDs

ids = np.arange(len(documents)).astype('int64')
index.add_with_ids(embeddings, ids)
print(f"Indexed {index.ntotal} vectors.")

```

We now have an index that can retrieve documents by their vector similarity. The IDs correspond to indices in the `documents` list (0 = first doc, etc.).

Step 3: Save the index to disk (optional). This is how you persist the index for later use:

```

faiss.write_index(index, "docs_index.faiss")
# ... later on, you can load it back:
index = faiss.read_index("docs_index.faiss")

```

This will save the entire index (including ID mappings) to a file. Ensure the file path has appropriate permissions; Faiss writes in a binary format.

Step 4: Query workflow. Now simulate a user query. We will embed the query, search the index for top-K passages, and then retrieve those passages.

```
# User query
query_text = "Where is the Eiffel Tower?"
# Step 4.1: Embed the query (again, using a random vector for demo)
query_vec = np.random.random((1, d)).astype('float32')
faiss.normalize_L2(query_vec) # normalize if using cosine

# Step 4.2: Search the index
k = 3
D, I = index.search(query_vec, k)
print("Retrieved IDs:", I)
print("Similarity scores:", D)
```

The `I` array contains the IDs of the top-3 matching documents, and `D` has their inner product scores (cosine similarities here, since we normalized). We can now fetch the actual text:

```
retrieved_docs = [documents[int(id)] for id in I[0] if id != -1]
print("Top retrieved documents:")
for doc in retrieved_docs:
    print(" -", doc)
```

We filter out `-1` just in case (Faiss uses -1 to denote invalid in case fewer than k results exist, but here we should have 3 results).

Finally, these `retrieved_docs` would be passed along with the query to an LLM or answer-generation module to produce a final answer. The generation part is outside Faiss's scope, but Faiss ensured we got relevant context.

Interpreting the example: If our embeddings were meaningful, we'd expect the query about "Eiffel Tower" to retrieve the passages about Paris and Eiffel Tower from our documents. Indeed, document indices likely to be retrieved (if the random vectors had any structure, which they don't, but assuming a real model) would be those containing "Eiffel Tower" or "Paris". In a real setup, you would see the Paris/Eiffel Tower sentences come up.

Batch Query Example: If we had multiple queries to process together, we could stack them in `query_vecs` of shape `(nq, d)` and call `search(query_vecs, k)`. Faiss will handle them in one go, possibly using threads for different queries.

Using Other Index Types: If our `documents` list was huge (say 100k entries), we might do something like:

```
index = faiss.IndexIVFFlat(faiss.IndexFlatIP(d), d, nlist=100,
faiss.METRIC_INNER_PRODUCT)
```

```
index.train(embeddings)
index.add(embeddings)
index.nprobe = 10
index = faiss.IndexIDMap(index) # IDMap after adding (though you could also
add_with_ids directly to IVF since it supports it)
```

And proceed similarly. The rest of the pipeline (embedding query, search, fetch) remains the same.

Performance and Best Practices

To wrap up, here are some additional tips and best practices when using Faiss-CPU:

- **Use appropriate data types:** Ensure your numpy arrays for vectors are `float32` (Faiss requires 32-bit floats for all operations) and `int64` for ids. Other dtypes will either error or produce incorrect results.
- **Memory alignment:** It's a good idea to use `np.ascontiguousarray` on your data before adding or searching, to avoid issues with Faiss reading non-contiguous memory. The Faiss Python wrappers often do this internally, but when in doubt, handle it explicitly.
- **Multi-threading control:** By default, Faiss will use multiple threads (via OpenMP) for many operations – e.g., it parallelizes distance computations across multiple CPU cores ⁷⁸ ⁷⁹. You can control this with:

```
faiss.omp_set_num_threads(n)
```

to set it to `n` threads. You might do this to limit resource usage (if running Faiss in a webserver with multiple processes, you might not want each query to use all cores). Conversely, if you find Faiss isn't using all cores (in some environments OpenMP might default to 1), explicitly set it to the number of cores you want to utilize.

- **Index selection and tuning:** Don't hesitate to refer to Faiss's own wiki for guidelines on choosing indexes ⁸¹ ⁸². For example, they suggest HNSW for high-accuracy when RAM is available, or IVF+PQ for very large scales ⁸³ ⁸⁴. In RAG, usually accuracy is paramount, so lean towards settings that favor accuracy (higher nprobe, higher efSearch, etc.) and test the impact on retrieval quality. You can use a small labeled set of queries to measure recall@k if possible.
- **Normalization and metrics:** As emphasized, use cosine similarity if that aligns with your embedding training. Almost all transformer-based text embeddings are cosine-based. Normalize vectors upfront or use an `IndexPreTransform` with `L2norm`. This ensures the distance (or dot) returned is meaningful. If you skip normalization, inner products might not correlate well unless the model inherently produces normalized outputs (some do, some don't).
- **Batch adds and queries:** Adding vectors in large batches (e.g., `index.add(large_array)`) is typically faster than many small `add()` calls due to reduced Python overhead and possibly better cache usage. Similarly, try to aggregate queries.
- **IVF partitioning:** If you use IVF, remember it works best when vectors are fairly uniformly spread among clusters. If your data has highly uneven cluster sizes, consider increasing nlist or using multi-level quantizers. Always train IVF on a large, representative sample of your data (or all data if feasible).

- **Maintaining ID mappings:** If you use `IndexIDMap`, keep in mind that if you remove vectors or add more, those changes need to be mirrored in whatever external storage of texts you have. One practice is to store texts in a dictionary keyed by ID, then you can retrieve by the ID directly. If you only ever append (no deletions), life is simple – you can just use array indices as ids. If you do remove, either handle the shifting (for flat) or use unique IDs that you retire on deletion.
- **Periodic Rebuilds vs Dynamic Update:** If your data changes often, sometimes rebuilding the index from scratch periodically (offline) and swapping it in can be simpler than many small insertions/deletions. For example, an IVF or HNSW index might be recreated nightly with new data. Faiss is optimized for search, not necessarily for heavy dynamic updates – though it can handle them in moderate amounts.
- **Distance interpretation:** For L2, the distances are squared Euclidean (if using `IndexFlatL2`). For inner product, if vectors are normalized, the range is $[-1, 1]$ for cosine. If not normalized, inner product could be unbounded based on norms. Sometimes people convert inner product to a similarity score (if needed for downstream).
- **Faiss version:** Keep an eye on Faiss releases. This guide is written around Faiss 1.7.x to 1.8.0 era. Newer versions might add features (like new index types, better pre/post processing, GPU enhancements we ignore here, etc.). Ensure compatibility of your index files with the runtime (indexes are generally backward-compatible within reason, but not guaranteed across major changes).
- **Thread safety:** Faiss indexes are mostly thread-safe for concurrent searches (multiple threads calling `search` on the same index) **as long as the index is not being modified** at the same time. Do not add or remove vectors concurrently with searching (or guard that with locks). If you need to handle simultaneous writes and reads, consider having a read-only query index and a separate process for building/updating indexes.
- **Monitoring and logging:** During development, monitor memory usage (some index types can consume more memory than just data, e.g., HNSW overhead or the direct map in IVF). Also, test query latency under load with realistic scenarios to ensure it meets your needs.
- **Similarity thresholding:** In some RAG systems, you might choose not to use extremely low similarity passages (because they might introduce noise to generation). You can apply a threshold on the distance or score. For instance, if using cosine, you might ignore results with $\text{cosine} < 0.2$ (just an example) even if they are top-k, and return fewer passages. Faiss won't do that for you automatically (unless you use `range_search`), but you can post-filter the results.

By following these practices, you can build a robust retrieval system using Faiss on CPU that efficiently supports your RAG pipeline.

Conclusion

We have covered the key components of using **Faiss-CPU** for vector search, from basic index types (Flat, IVF, HNSW, etc.) and their usage to advanced features like ID mapping, range search, and filtered search. We also walked through converting GPU-based code to CPU and considered how to choose and configure indexes for typical RAG scenarios focusing on text embeddings. Finally, we demonstrated an end-to-end example of building a small Faiss index for text and querying it, and provided tips to ensure performance and reliability.

With this knowledge, you should be able to implement a Faiss-backed retrieval system using the official `faiss-cpu` package, configure it for your dataset size and accuracy needs, and integrate it into a pipeline

for retrieval-augmented generation or similar tasks. Faiss on CPU is powerful and often sufficient for many applications – and if needs grow, you can later consider scaling out or switching to faiss-gpu while keeping the core concepts the same. Good luck with your implementation!

Sources:

- Faiss Wiki – *Guidelines for choosing an index* 4 23
 - Faiss Wiki – *Flat vs IVF vs PQ descriptions* 85 86
 - Faiss Wiki – *HNSW specifics* 22 26
 - Faiss Wiki – *IDMapping and Preprocessing* 31 36
 - Faiss Wiki – *IndexRefineFlat usage* 45 46
 - Faiss Wiki – *Removal and IDSelector details* 63 75
 - BGE Tutorial – *Overview of Flat, IVF, HNSW indexes* 2 87
 - Faiss Documentation – *ParameterSpace usage* 53 58
 - Medium Article – *Faiss indexing and multi-threading* 79 78
-

1 2 10 13 20 24 27 28 29 30 87 **Faiss Indexes — BGE documentation**

https://bge-model.com/tutorial/3_Indexing/3.1.3.html

3 7 8 9 12 14 19 21 22 26 85 86 **Faiss indexes · facebookresearch/faiss Wiki · GitHub**

<https://github.com/facebookresearch/faiss/wiki/Faiss-indexes>

4 23 25 81 82 83 84 **Guidelines to choose an index · facebookresearch/faiss Wiki · GitHub**

<https://github.com/facebookresearch/faiss/wiki/Guidelines-to-choose-an-index>

5 17 18 78 79 **Understanding FAISS Indexing. In this article we will dive deep into... | by Tanya Soni | Medium**

<https://arithmancylabs.medium.com/understanding-faiss-indexing-86ec98048bd9>

6 35 51 52 63 64 65 66 **Special operations on indexes · facebookresearch/faiss Wiki · GitHub**

<https://github.com/facebookresearch/faiss/wiki/Special-operations-on-indexes>

11 31 32 34 36 37 39 40 41 42 45 46 49 50 **Pre and post processing · facebookresearch/faiss Wiki · GitHub**

<https://github.com/facebookresearch/faiss/wiki/Pre--and-post-processing>

15 16 **Effortless large-scale image retrieval with FAISS: A hands-on tutorial | UnfoldAI**

<https://unfoldai.com/effortless-large-scale-image-retrieval-with-faiss-a-hands-on-tutorial/>

33 **llm/Lib/site-packages/faiss/class_wrappers.py · Draken007/geochatbot at 78d10330a3475be907a3653b2973e0cd8cbf3256**

https://huggingface.co/spaces/Draken007/geochatbot/blob/78d10330a3475be907a3653b2973e0cd8cbf3256/llm/Lib/site-packages/faiss/class_wrappers.py

38 43 44 **The index factory · facebookresearch/faiss Wiki · GitHub**

<https://github.com/facebookresearch/faiss/wiki/The-index-factory>

47 **Struct faiss::IndexRefineFlat - Faiss Documentation**

https://faiss.ai/cpp_api/struct/structfaiss_1_1IndexRefineFlat.html

48 **Struct faiss::IndexRefine - Faiss Documentation**

https://faiss.ai/cpp_api/struct/structfaiss_1_1IndexRefine.html

53 54 55 60 61 62 **Index IO, cloning and hyper parameter tuning · facebookresearch/faiss Wiki · GitHub**

<https://github.com/facebookresearch/faiss/wiki/Index-IO,-cloning-and-hyper-parameter-tuning>

56 57 58 59 67 68 69 70 71 72 73 74 75 76 **Setting search parameters for one query · facebookresearch/faiss Wiki · GitHub**

<https://github.com/facebookresearch/faiss/wiki/Setting-search-parameters-for-one-query>

77 **Enhancing GPU-Accelerated Vector Search in Faiss with NVIDIA cuVS**

<https://developer.nvidia.com/blog/enhancing-gpu-accelerated-vector-search-in-faiss-with-nvidia-cuvs/>

80 **GPU Faiss with cuVS · facebookresearch/faiss Wiki · GitHub**

<https://github.com/facebookresearch/faiss/wiki/GPU-Faiss-with-cuVS>